Milestone 1 | Project Charter & Scope

Team roster

Aniket Chaudhari | Project Lead | chaudhari.78@osu.edu

I will manage the timeline, maintain the GitHub repo, and check that the team's work runs together.

Sophiya Khurana | Data Engineer / Writer | khurana.65@osu.edu

I will clean the review data, check for missing values and duplicates, and write the data quality section.

Ethan Brown | Data Engineer / Writer | brown.9058@osu.edu

I will analyze the label counts and review lengths, make the charts, and write the dataset summary.

Lianyu Yang | App Engineer | yang.7674@osu.edu

I will build and deploy the Streamlit app, connect the trained models, and write the app instructions.

Shilin Dong | ML Engineer | dong.1301@osu.edu

I will train and tune the models, compare their results, and write the model results section.

Problem statement:

Our system will include two classical ML models and one deep learning or modern NLP model on a 50,000-review sample, including data like product id, summary, review text, etc. The primary inputs are the review text and summary fields. The primary target is the rating, positive, negative, or neutral.

The system offers rating prediction for those websites that have reviews but not ratings. The users can feed the reviews into our application and get rating predictions for making purchasing decisions.

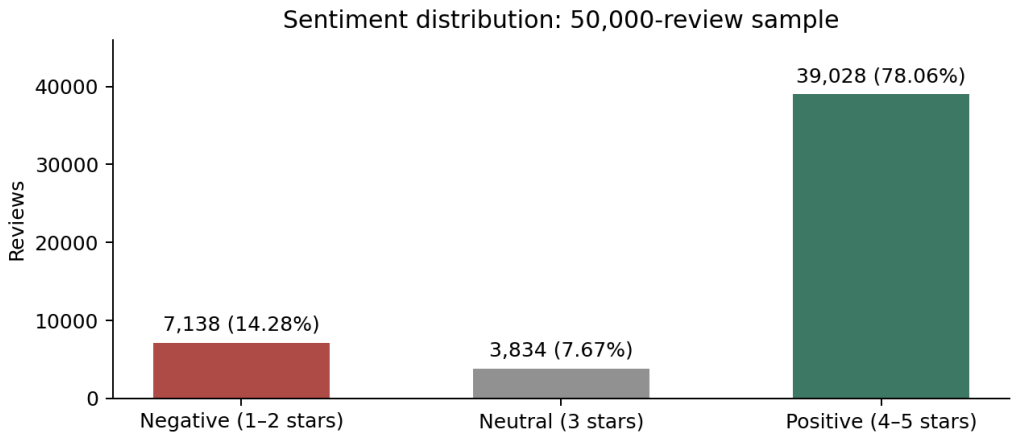
Dataset summary:

Source and sample: We randomly selected 50,000 reviews from the full Amazon Fine Food Reviews CSV, using seed 42 and sampling without replacement. This is a team-created sample; permission to use it instead of the course's shared sample is still to be confirmed. The figures below describe the sample before cleaning.

Labels: Negative (1–2 stars): 7,138 reviews (14.28%); neutral (3 stars): 3,834 (7.67%); positive (4–5 stars): 39,028 (78.06%). Positive reviews make up most of the sample, so we will report per-class results and macro-F1 alongside accuracy.

Review length: The average review contains ~80 words, counted by splitting the Text field on whitespace.

Data quality: No fields are empty or contain only whitespace. Five summaries contain “N/A” and need review. There are 3,674 repeated Summary. Text pairs after ignoring capitalization and extra whitespace. We will remove repeated content before splitting the data. All star ratings are valid.



Used ChatGPT for bar graph: prompt: Make a score distribution bar chart from the randomly selected kaggle dataset.

Technical approach:

Data: We will combine the review summary and text, remove HTML tags, and clean up spacing. We will keep words such as “not.” Ratings of 1–2 stars will be negative, 3 stars neutral, and 4–5 stars positive. We will remove repeated reviews before splitting the data.

Component	Choice	Reason
Feature representation 1	Bag of Words	Counts each word.
Feature representation 2	TF-IDF	Downweights common words.
Classical model 1	Multinomial Naive Bayes	Fast baseline for text.
Classical model 2	Logistic Regression	Learns weights for words.
Neural model	Feedforward neural network	Learns nonlinear patterns.

Features: We will use Bag of Words and TF-IDF from Lecture 6. We will test single words and two-word phrases, such as “not good,” and limit the vocabulary to keep training manageable.

Models: We will use Naive Bayes from Lecture 9, Logistic Regression, and a neural network with two hidden layers. We will tune settings on validation data and stop neural training when validation loss stops improving. Lessons for Logistic Regression and neural networks not covered yet so no references.

Evaluation: All models will use the same 65% training, 15% validation, and 20% test split, with seed 42 and similar class proportions in each split. We will fit text features on training data only and use the test set after tuning. We will report accuracy, precision, recall, macro-F1, per-class scores, confusion matrices, and training time. We will compare against always predicting the largest class and plot the neural network's training and validation loss.

App: We will deploy a Streamlit app on Streamlit Community Cloud. Users will enter a review and see predictions from at least two models side by side. Each saved model will use the same text preprocessing it used during training.

Project timeline:

Week	Work	Who does what
Nov 3–8 Milestone 1	Check data, confirm labels, and submit the charter.	Aniket: charter/repo. Sophiya + Ethan: data summary. Shilin: model plan. Lianyu: app plan.
Nov 9–15 Milestone 2	Build text features, train a baseline, and review errors.	Sophiya + Ethan: features/errors. Shilin: Naive Bayes. Lianyu: app draft. Aniket: combine work.
Nov 16–22 Milestone 3	Train all models, compare results, and finish the app prototype.	Shilin + Aniket: models/evaluation. Sophiya + Ethan: charts/write-up. Lianyu: app.
Nov 23–29 Milestone 4	Deploy the app and submit the report and slides.	Lianyu: deploy. Sophiya + Ethan: report. Shilin: results. Aniket: final checks. All: slides/review.
Nov 30–Dec 2	Rehearse and prepare for questions. Check Carmen for the demo date.	Everyone: practice their section and explain the models.

Risk: Neural-network training or app deployment could take longer than expected.

Mitigation: Shilin will test a small network early. Lianyu will test deployment by Nov 22. We will reduce the network size if training is too slow and keep the classical models ready for the app while finishing the neural comparison. Aniket will check progress each week.

GitHub repository:

https://github.com/anik1617/group_omega_3521